

Machine Learning

CSE343/CSE543/ECE363/ECE563

Lecture 11 | Take your own notes during lectures

Vinayak Abrol <abrol@iiitd.ac.in>



INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY **DELHI**



Primal and Dual Problem

$$\begin{array}{ll}\min_w & f(w) \\ \text{s.t.} & g_i(w) \leq 0, \quad i = 1, \dots, k \\ & h_i(w) = 0, \quad i = 1, \dots, l.\end{array}$$

$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_{i=1}^k \alpha_i g_i(w) + \sum_{i=1}^l \beta_i h_i(w).$$

Primal

$$\min_w \theta_{\mathcal{P}}(w) = \min_w \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta)$$

$$\theta_{\mathcal{P}}(w) = \begin{cases} f(w) & \text{if } w \text{ satisfies primal constraints} \\ \infty & \text{otherwise.} \end{cases}$$

Dual

$$\max_{\alpha, \beta: \alpha_i \geq 0} \theta_{\mathcal{D}}(\alpha, \beta) = \max_{\alpha, \beta: \alpha_i \geq 0} \min_w \mathcal{L}(w, \alpha, \beta)$$

$$d^* = \max_{\alpha, \beta: \alpha_i \geq 0} \min_w \mathcal{L}(w, \alpha, \beta) \leq \min_w \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta) = p^*$$

Primal and Dual Problem

$$\begin{aligned} \min_w \quad & f(w) \\ \text{s.t.} \quad & g_i(w) \leq 0, \quad i = 1, \dots, k \\ & h_i(w) = 0, \quad i = 1, \dots, l. \end{aligned}$$

Primal

$$\min_w \theta_{\mathcal{P}}(w) = \min_w \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta)$$

$$\theta_{\mathcal{P}}(w) = \begin{cases} f(w) & \text{if } w \text{ satisfies primal constraints} \\ \infty & \text{otherwise.} \end{cases}$$

$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_{i=1}^k \alpha_i g_i(w) + \sum_{i=1}^l \beta_i h_i(w).$$

Dual

$$\begin{aligned} \frac{\partial}{\partial w_i} \mathcal{L}(w^*, \alpha^*, \beta^*) &= 0, \quad i = 1, \dots, n \\ \frac{\partial}{\partial \beta_i} \mathcal{L}(w^*, \alpha^*, \beta^*) &= 0, \quad i = 1, \dots, l \\ \alpha_i^* g_i(w^*) &= 0, \quad i = 1, \dots, k \\ g_i(w^*) &\leq 0, \quad i = 1, \dots, k \\ \alpha^* &\geq 0, \quad i = 1, \dots, k \end{aligned}$$

$d^* = p^*$ when (w, α, β) satisfy
Karush-Kun-Tucker Conditions

$$d^* = \max_{\alpha, \beta: \alpha_i \geq 0} \min_w \mathcal{L}(w, \alpha, \beta) \leq \min_w \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta) = p^*$$

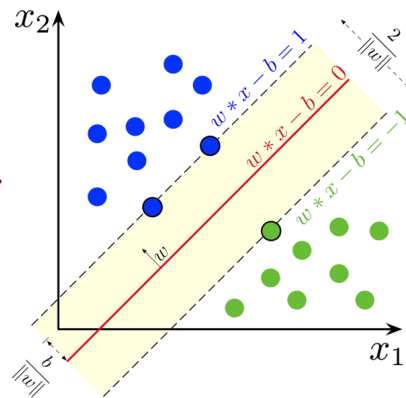
SVM: The Optimal Margin Classifier

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j (x^{(i)})^T x^{(j)} - b \sum_{i=1}^m \alpha_i y^{(i)}$$

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j (x^{(i)})^T x^{(j)}.$$

KKT conditions are satisfied.
Verify!

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle. \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, m \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0, \end{aligned}$$



Consider a test example with $y=1$. Here we use the value of w in terms of α

$$\begin{aligned} w^T x + b &= \left(\sum_{i=1}^m \alpha_i y^{(i)} x^{(i)} \right)^T x + b \\ &= \sum_{i=1}^m \alpha_i y^{(i)} \langle x^{(i)}, x \rangle + b. \end{aligned}$$

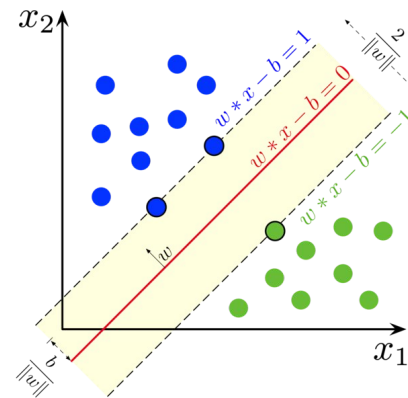
Making a prediction requires the inner-product $\langle x^{(i)}, x \rangle$
Multiplied with $\alpha_i > 0$ i.e., support vectors.

The SMO (sequential minimal optimization) algorithm by John Platt is used for solving SVM

SVM: Why to even care about margins ?

If our hypothesis (value) is near the decision boundary of decision space, we don't necessarily learn much from our mistakes.

If we're far away from any boundary, we have more room for mistakes.



SVM: Why to even care about margins ?

If our hypothesis (value) is near the decision boundary of decision space, we don't necessarily learn much from our mistakes.

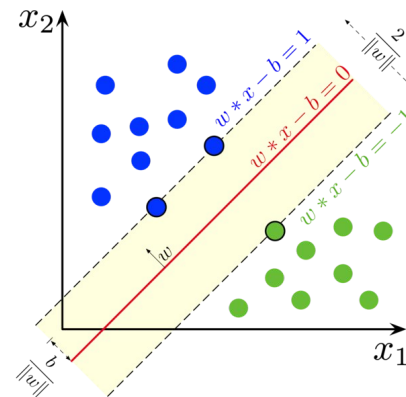
If we're far away from any boundary, we have more room for mistakes.

Occam's razor:

"when you have two competing theories that make exactly the same predictions, the simpler one is the better."

Why? Scarcity of simple classifiers than complicated ones.

Probability of finding a really good fit by accident is far less.



SVM: Why to even care about margins ?

If our hypothesis (value) is near the decision boundary of decision space, we don't necessarily learn much from our mistakes.

If we're far away from any boundary, we have more room for mistakes.

Occam's razor:

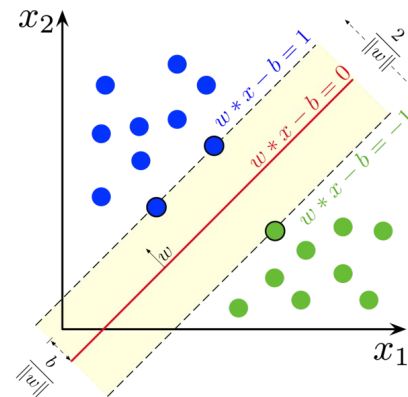
"when you have two competing theories that make exactly the same predictions, the simpler one is the better."

Why? Scarcity of simple classifiers than complicated ones.

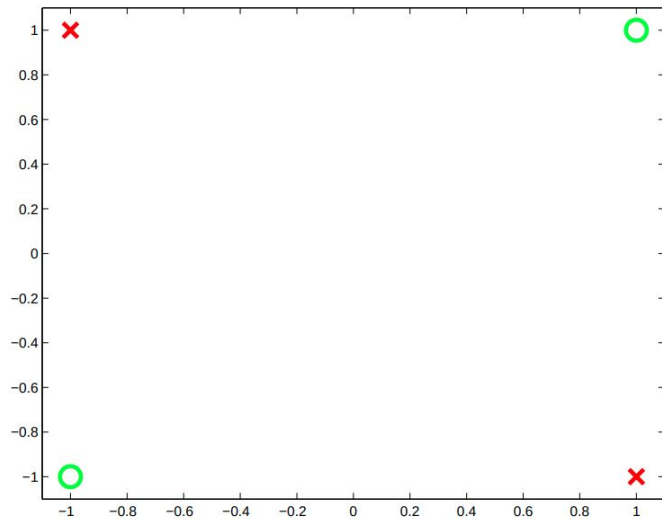
Probability of finding a really good fit by accident is far less.

"Simple" can mean:

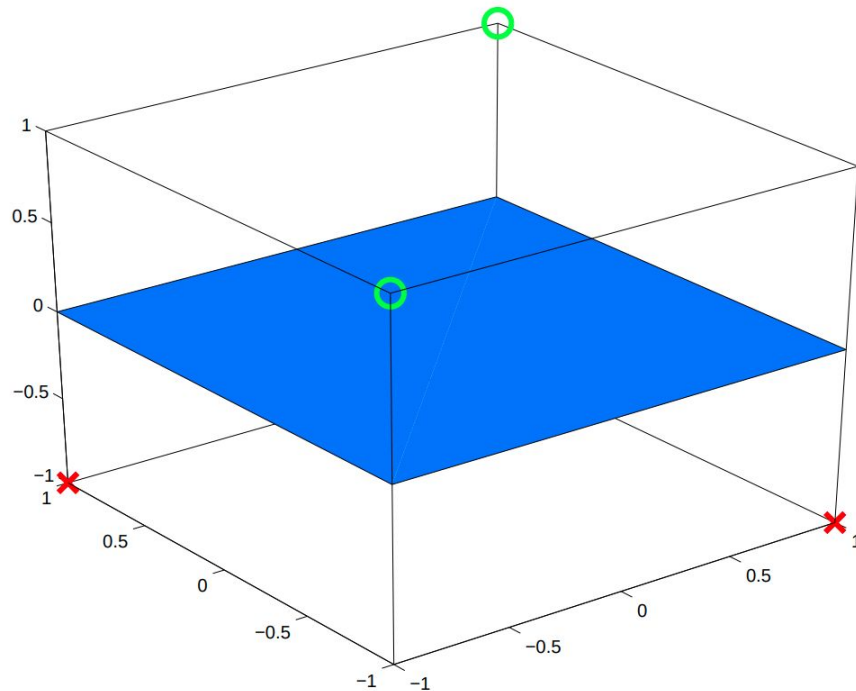
- Low-dimensional
- Large margin (large margins make simple classifiers)
- Low complexity



Non-Linear Problems: Existence of Simple Classifier

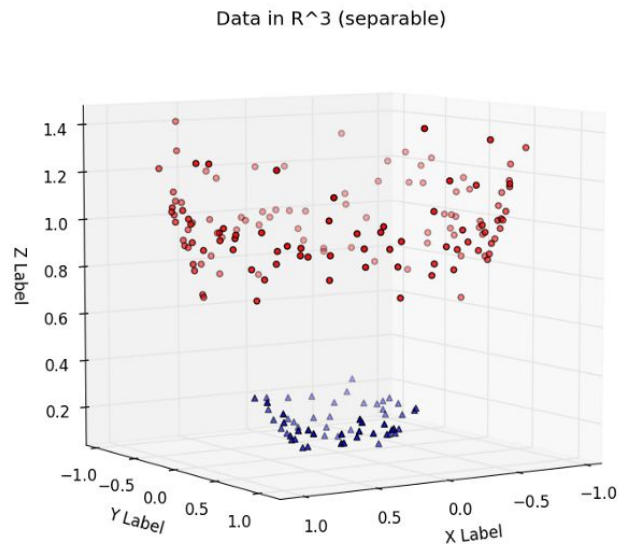
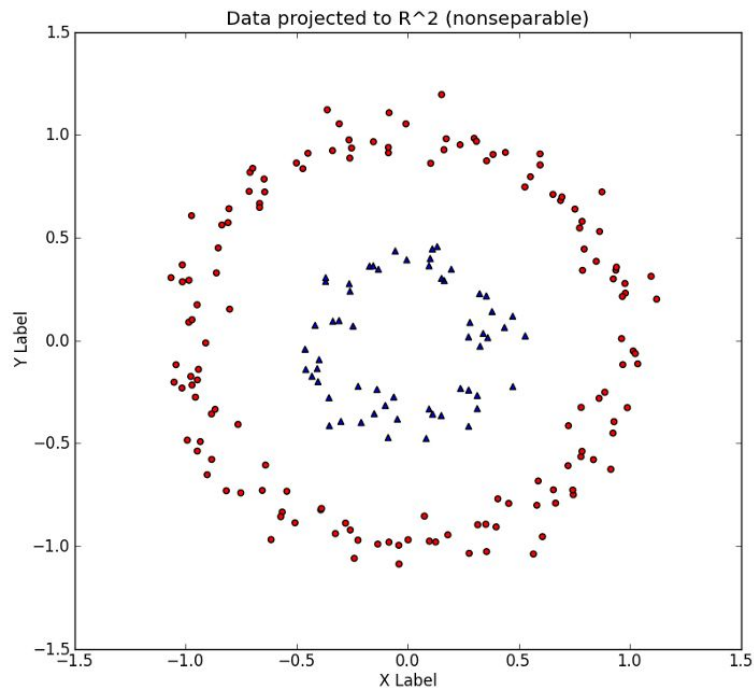


x, y



x, y, xy

Non-Linear Problems: Existence of Simple Classifier



Kernel Trick and Feature Expansion

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle. \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, m \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0, \end{aligned}$$

Kernel Trick and Feature Expansion

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle. \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, m \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0, \end{aligned}$$

We can work in transform domain usually defined by Φ

Use $\Phi(x)$ instead of x

Kernel Trick and Feature Expansion

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle. \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, m \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0, \end{aligned}$$

We can work in transform domain usually defined by Φ

Use $\Phi(x)$ instead of x

Trust Me! SVM won't break and all equations are intact.

But remember there is an inner-product (and we really care about it) so let's define it as

$$K(x, z) = \phi(x)^T \phi(z)$$

Kernel Trick and Feature Expansion

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle. \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, m \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0, \end{aligned}$$

We can work in transform domain usually defined by Φ
Use $\Phi(x)$ instead of x

Trust Me! SVM won't break and all equations are intact.

But remember there is an inner-product (and we really care about it) so let's define it as

$$K(x, z) = \phi(x)^T \phi(z)$$

But what about $\Phi(x)$

We typically choose Φ so as to work in higher dimensions.

This means we are expanding the feature space. $\Phi(x) = x'$ is now a big vector.

So far we have:

Large Margin and Simple Classifier

Kernel Trick and Feature Expansion

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle. \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, m \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0, \end{aligned}$$

We can work in transform domain usually defined by Φ
Use $\Phi(x)$ instead of x

Trust Me! SVM won't break and all equations are intact.

But remember there is an inner-product (and we really care about it) so let's define it as

$$K(x, z) = \phi(x)^T \phi(z)$$

But what about $\Phi(x)$

We typically choose Φ so as to work in higher dimensions.

This means we are expanding the feature space. $\Phi(x) = x'$ is now a big vector.

So far we have:

Large Margin and Simple Classifier

And in addition all we require is low complexity ! Sweet Spot

Which leads to the so called world of
Kernel Methods

Kernel Trick and Feature Expansion

$$K(x, z) = \phi(x)^T \phi(z)$$

$$\phi(x) = \begin{bmatrix} x_1x_1 \\ x_1x_2 \\ x_1x_3 \\ x_2x_1 \\ x_2x_2 \\ x_2x_3 \\ x_3x_1 \\ x_3x_2 \\ x_3x_3 \\ \sqrt{2c}x_1 \\ \sqrt{2c}x_2 \\ \sqrt{2c}x_3 \\ c \end{bmatrix}$$

Kernel Trick and Feature Expansion

$$K(x, z) = \phi(x)^T \phi(z)$$

$$\phi(x) = \begin{bmatrix} x_1x_1 \\ x_1x_2 \\ x_1x_3 \\ x_2x_1 \\ x_2x_2 \\ x_2x_3 \\ x_3x_1 \\ x_3x_2 \\ x_3x_3 \\ \sqrt{2cx_1} \\ \sqrt{2cx_2} \\ \sqrt{2cx_3} \\ c \end{bmatrix}$$

$$\begin{aligned} K(x, z) &= (x^T z + c)^2 \\ &= \sum_{i,j=1}^n (x_i x_j)(z_i z_j) + \sum_{i=1}^n (\sqrt{2cx_i})(\sqrt{2cz_i}) + c^2 \end{aligned}$$

Kernel Trick and Feature Expansion

$$K(x, z) = \phi(x)^T \phi(z)$$

$$K_{ij} = K(x^{(i)}, x^{(j)}) = \phi(x^{(i)})^T \phi(x^{(j)}) = \phi(x^{(j)})^T \phi(x^{(i)}) = K(x^{(j)}, x^{(i)}) = K_{ji},$$

K must be symmetric

$$\phi(x) = \begin{bmatrix} x_1x_1 \\ x_1x_2 \\ x_1x_3 \\ x_2x_1 \\ x_2x_2 \\ x_2x_3 \\ x_3x_1 \\ x_3x_2 \\ x_3x_3 \\ \sqrt{2cx_1} \\ \sqrt{2cx_2} \\ \sqrt{2cx_3} \\ c \end{bmatrix}$$

$$\begin{aligned} K(x, z) &= (x^T z + c)^2 \\ &= \sum_{i,j=1}^n (x_i x_j)(z_i z_j) + \sum_{i=1}^n (\sqrt{2cx_i})(\sqrt{2cz_i}) + c^2 \end{aligned}$$

$$K(x, z) = (x^T z + c)^d$$

$O(n^d)$ -dimensional space, computing $K(x, z)$ still takes only $O(n)$ time

Kernel Trick and Feature Expansion

$$K(x, z) = \phi(x)^T \phi(z)$$

$$K_{ij} = K(x^{(i)}, x^{(j)}) = \phi(x^{(i)})^T \phi(x^{(j)}) = \phi(x^{(j)})^T \phi(x^{(i)}) = K(x^{(j)}, x^{(i)}) = K_{ji},$$

K must be symmetric

$$\phi(x) = \begin{bmatrix} x_1x_1 \\ x_1x_2 \\ x_1x_3 \\ x_2x_1 \\ x_2x_2 \\ x_2x_3 \\ x_3x_1 \\ x_3x_2 \\ x_3x_3 \\ \sqrt{2cx_1} \\ \sqrt{2cx_2} \\ \sqrt{2cx_3} \\ c \end{bmatrix}$$

$$\begin{aligned} K(x, z) &= (x^T z + c)^2 \\ &= \sum_{i,j=1}^n (x_i x_j)(z_i z_j) + \sum_{i=1}^n (\sqrt{2cx_i})(\sqrt{2cx_i}) + c^2 \end{aligned}$$

The basic idea of Kernel Trick is that we should be able to compute the inner-product in higher-dimensional space Φ without even doing any transformation of x

$$K(x, z) = (x^T z + c)^d$$

Note: often $K(.,.)$ is used to denote both kernel function and kernel matrix

$O(n^d)$ -dimensional space, computing $K(x, z)$ still takes only $O(n)$ time

Valid Kernels

$$K(x, z) = \phi(x)^T \phi(z)$$

A Kernel is valid if there exist a inner-product via Φ .

Valid Kernels

$$K(x, z) = \phi(x)^T \phi(z)$$

A Kernel is valid if there exist a inner-product via Φ .

And if a kernel is valid we can show that

$$\begin{aligned} z^T K z &= \sum_i \sum_j z_i K_{ij} z_j \\ &= \sum_i \sum_j z_i \phi(x^{(i)})^T \phi(x^{(j)}) z_j \\ &= \sum_i \sum_j z_i \sum_k \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\ &= \sum_k \sum_i \sum_j z_i \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\ &= \sum_k \left(\sum_i z_i \phi_k(x^{(i)}) \right)^2 \\ &\geq 0. \end{aligned}$$

Valid Kernels

$$K(x, z) = \phi(x)^T \phi(z)$$

A Kernel is valid if there exist a inner-product via Φ .

And if a kernel is valid we can show that

$$\begin{aligned} z^T K z &= \sum_i \sum_j z_i K_{ij} z_j \\ &= \sum_i \sum_j z_i \phi(x^{(i)})^T \phi(x^{(j)}) z_j \\ &= \sum_i \sum_j z_i \sum_k \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\ &= \sum_k \sum_i \sum_j z_i \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\ &= \sum_k \left(\sum_i z_i \phi_k(x^{(i)}) \right)^2 \\ &\geq 0. \end{aligned}$$

Mercer Kernel/Theorem: if $K(.,.)$ is positive semi-definite Φ exists.

In many cases there is a simple expression for K even if there isn't one for Φ

In fact, it sometimes happens that we know K without knowing Φ

Valid Kernels

$$K(x, z) = \phi(x)^T \phi(z)$$

A Kernel is valid if there exist a inner-product via Φ .

Mercer Kernel/Theorem: if $K(.,.)$ is positive semi-definite Φ exists.

And if a kernel is valid we

$$\begin{aligned} z^T K z &= \sum_i \sum_j z_i K_{ij} z_j \\ &= \sum_i \sum_j z_i \phi(x^{(i)})^T \phi(x^{(j)}) \\ &= \sum_i \sum_j z_i \sum_k \phi_k(x^{(i)}) \phi_k(x^{(j)}) \\ &= \sum_k \sum_i \sum_j z_i \phi_k(x^{(i)}) \phi_k(x^{(j)}) \\ &= \sum_k \left(\sum_i z_i \phi_k(x^{(i)}) \right)^2 \\ &\geq 0. \end{aligned}$$

Parameters::

C : float, default=1.0

Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared l2 penalty.

kernel : {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} or callable, default='rbf'

Specifies the kernel type to be used in the algorithm. If none is given, 'rbf' will be used. If a callable is given it is used to pre-compute the kernel matrix from data matrices; that matrix should be an array of shape (n_samples, n_samples).

degree : int, default=3

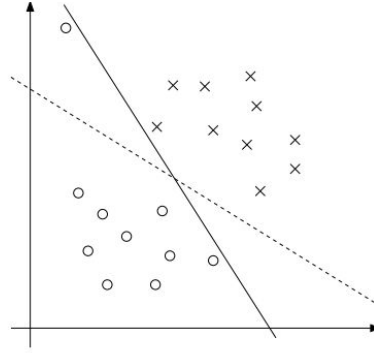
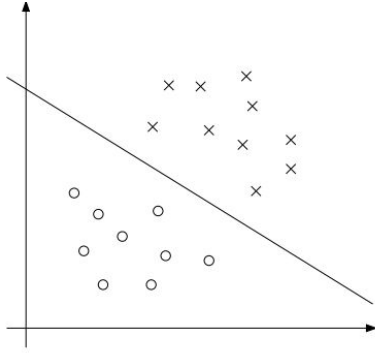
Degree of the polynomial kernel function ('poly'). Ignored by all other kernels.

gamma : {'scale', 'auto'} or float, default='scale'

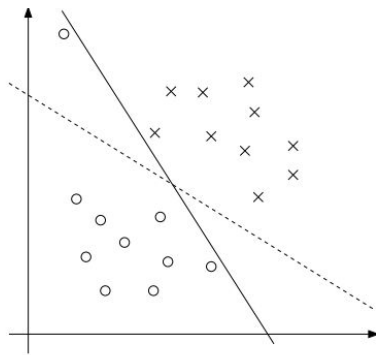
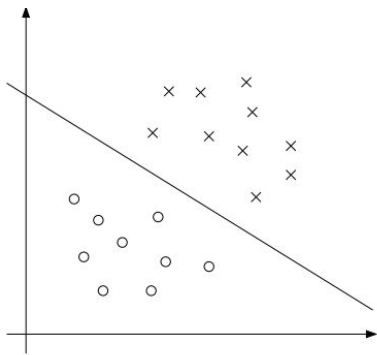
Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.

- if gamma='scale' (default) is passed then it uses 1 / (n_features * X.var()) as value of gamma,
- if 'auto', uses 1 / n_features.

SVM: Regularization



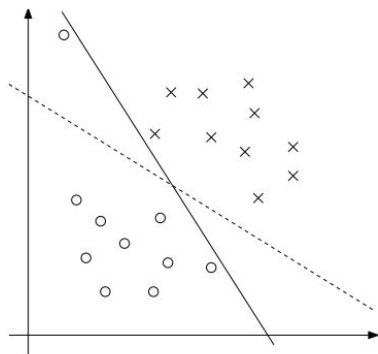
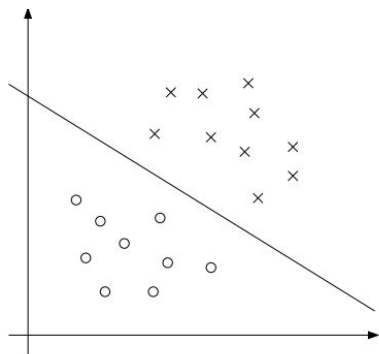
SVM: Regularization



$$\begin{aligned} \min_{\gamma, w, b} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i \\ \text{s.t.} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \dots, m \\ & \xi_i \geq 0, \quad i = 1, \dots, m. \end{aligned}$$

Suppose an example has functional margin $(1 - \xi_i)$, we would pay a cost of the objective function being increased by $C\xi_i$

SVM: Regularization



$$\begin{aligned} \min_{\gamma, w, b} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i \\ \text{s.t.} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \dots, m \\ & \xi_i \geq 0, \quad i = 1, \dots, m. \end{aligned}$$

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, m \\ & \sum_{i=1}^m \alpha_i y^{(i)} = 0, \end{aligned}$$

Suppose an example has functional margin $(1 - \xi_i)$, we would pay a cost of the objective function being increased by $C\xi_i$

The parameter C controls the relative weighting between the twin goals of making the $\|w\|^2$ small enough so as to ensure that most examples have functional margin at least 1.

Finally!

